✅ **Task #1: Debug the Agent**

Problem

The agent was able to:

- Read input file
- Generate summary

But failed to:

- Write summary to output file ❌

Root Cause

The issue was **not in the write_to_file function**, but in the **Agent implementation**:

- Tools were passed incorrectly as metadata instead of functions
- OpenAI function calling was not properly handled
- No execution logic for tool calls (write_to_file never executed)

Fix Implemented

1. Correct Tool Passing

```
tools = [read_from_file, write_to_file]
```

2. Implement Tool Schema Mapping

```
def _get_tool_schema(self):
    return [tool._tool_metadata for tool in self.tools]
```

3. Implement Tool Execution Logic

```
def _execute_tool(self, tool_name, arguments):
    for tool in self.tools:
        if tool._tool_metadata["function"]["name"] == tool_name:
            return tool(**arguments)
```

4. Handle Function Calls from OpenAI

- Detect tool_calls
- Execute corresponding Python function
- Return results through broker

5. Improve Prompt Instructions

Updated prompt to explicitly enforce tool usage:

- Read file → Summarize → Write file
- Mandatory write_to_file call

✅ **Result**

- Agent successfully reads input
- Generates summary
- Writes output file

Output Generated

resources/output/function_calling_summary.md

✅ **Task #2: Implement list_directory**

Problem

The list_directory function in file_ops.py was incomplete:

- ❌ Function contained only pass
- ❌ Agent could not list files from a directory
- ❌ No support for filtering files by extension

Root Cause

The function was not implemented, so:

- The agent had no capability to explore directories
- Any attempt to call list_directory would fail or return nothing
- The tool was defined but not usable

Fix Implemented

1. Validate Directory Path

```
path = Path(directory_path).resolve()
```

- Ensures absolute path resolution
- Standardizes file handling
-

2. Check Directory Existence

```
if not path.exists():
```

- Prevents invalid path errors
- Returns structured error response

3. Ensure Path is a Directory

```
if not path.is_dir():
```

- Avoids treating files as directories
- Improves robustness

4. List Files in Directory

```
for item in path.iterdir():
    if item.is_file():
```

- Iterates through directory contents
- Filters only files (excludes folders)

5. Optional File Extension Filtering

```
if file_extension:
    if item.suffix == file_extension:
        files.append(item.name)
```

- Supports filtering like .md, .txt
- Enhances flexibility

6. Return Structured Output

```
return {
    "status": "success",
```

```
    "files": files,
    "total_count": len(files),
    "directory_path": str(path)
}
```

- Consistent with other tools
- Easy for agent to interpret

✅ **Result**

- ✓ Successfully lists files in a directory
- ✓ Supports optional filtering by file extension
- ✓ Returns structured and consistent response
- ✓ Fully integrated as a usable tool for the agent